

Gruppe 1

Capstone project

Introduksjon

- Søkbar database med trafikk, sykkel og værdata
- Hensikten er å kunne undersøke sammenhenger mellom værforhold, sykkelvaner og trafikkvaner.
- Det er ambisjoner om å utvikle en modell som kan brukes i flere norske byer og eventuelt andre land.
- Et sentralt formål er å undersøke om vær påvirker sykkel og bilvaner.

Info om Data

- Vi har valgt å se på trafikk og værdata for Fredrikstad, med fokus på sammenheng og korrelasjon mellom disse dataene. Det er utforsket og sett etter trender og tendenser i dataene.
- Det er fokusert på data fra Fredrikstad, og i hovedsak de store innfartsveiene og utfartsveiene fra rundt sentrumsområdet.
- Værdata er hentet fra målestasjonen Strømtangen fyr da dette var det eneste målepunktet i Fredrikstad med tilsynelatende tilstrekkelig data.
- Trafikkdata er hentet fra Statens Vegvesen på <https://trafikkdata.atlas.vegvesen.no/kart>
- Værdata er hentet fra Norsk Klimaservicesenter på <https://klimaservicesenter.no/>



Statens vegvesen

NORSK KLIMASERVICESENTER



**Meteorologisk
institutt**

Trafikkdata

Navn	Verdi	Forklaring	Navn	Verdi	Forklaring
Trafikkregistreringspunkt	string	Id nummer på målestasjonen trafikkdataene er registrert.	Felt	string	Hvilke kjørefelt målingen gjelder.
Navn	string	Navnet på trafikkregistreringspunktet.	Trafikkmengde	string	Antall passeringer i måleperioden.
Vegreferanse	string	Hvilken vei stasjonen ligger ved. Gitt i statens vegvesens veinummerering.	Dekningsgrad	string	Andel av tidsintervall registreringen har data for, gitt i prosent.
Fra	string	Dato og tidspunkt for målingens start.	Antall timer total	integer	Totalt antall timer i måleperioden.
Til	string	Dato og tidspunkt for målingens slutt.	Antall timer inkludert	integer	Totalt antall timer som har data i måleperioden.
Dato	date	Dato for målingen.	Antall timer ugyldig	integer	Antall timer i perioden med ukjent kvalitet.
Fra tidspunkt	timestamp	Tidspunkt for måleperiodens start.	Ikke gyldig lengde	string	Antall kjøretøy med underkjent/ukjent lengde.
Til tidspunkt	timestamp	Tidspunkt for måleperiodens slutt.	Lengdekvalitetsgrad	string	Andel kjøretøy med godkjent lengdemåling, gitt i prosent.

Trafikkdata forts.

Navn	Verdi	Forklaring
< 5,6 m	string	Antall kjøretøy kortere enn 5,6 meter.
≥ 5,6 m	string	Antall kjøretøy lengre enn eller lik 5,6 meter.
5,6m – 7,6m	string	Antall kjøretøy mellom 5,6 og 7,6 meter.
7,6m – 12,5m	string	Antall kjøretøy mellom 7,6 og 12,5 meter.
12,5m – 16,0m	string	Antall kjøretøy mellom 12,5 og 16 meter.
≥ 16,0m	string	Antall kjøretøy lengre enn eller lik 16 meter.
16,0m – 24,0m	string	Antall kjøretøy mellom 16 og 24 meter.
≥ 24,0m	string	Kjøretøy lengre enn eller lik 24 meter (gyldig opp til 27 m).

Værdata

Navn	Verdi	Forklaring
Navn	string	Navnet på målestasjon.
Stasjon	string	Stasjons ID.
Tid (norsk normaltid)	string	Registreringstidspunkt for måleperioden. Hver måleperiode er på én time.
Nedbør (1 t)	string	Nedbør siste time, gitt i millimeter.
Lufttemperatur	string	Temperatur på luft i måletidspunktet.
Antall minutter nedbør siste time	string	Antall minutter det er registrert nedbør den siste timen.
Høyeste middelvind (1 t)	string	Høyeste gjennomsnitt av vindhastighet over en ti-minutters periode den siste timen. Høyeste gjennomsnitt av seks stykk ti-minutters perioder.

Hypoteser

1. Den totale trafikkmengden inn og ut av Fredrikstad gjennom én time er høyest i rushtiden hverdager mellom 15 : 00 – 16 : 00.
2. Den totale bil trafikkmengden inn og ut av Fredrikstad er lavere i juli måned enn for resten av året.
3. Det finnes en positiv korrelasjon for temperatur og total sykkeltrafikk i Fredrikstad målt over en referanseperiode mellom 10:00 – 14:00.
4. Det finnes en negativ korrelasjon for temperatur og total biltrafikk i Fredrikstad målt over en referanseperiode mellom 10:00 – 14:00.

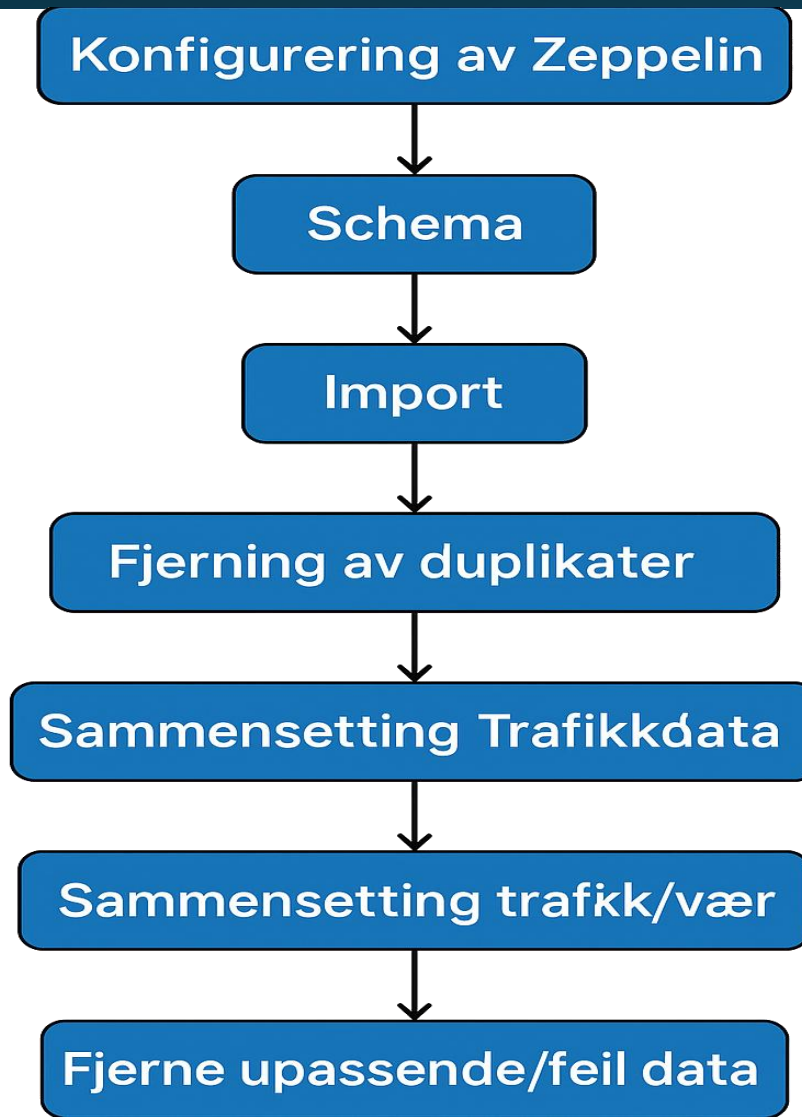


https://png.pngtree.com/png-vector/20221018/ourmid/pngtree-workers-organizing-data-storage-flat-vector-illustration-png-image_6321237.png



https://blogger.googleusercontent.com/img/b/R29vZ2xl/AVvXsEgt38F78lbRW1Am0jP3_GZGgpluZI8HZ2E9b8eNyyvioACDEoivyUH023kLpr6UUt641zfqKSGFgCxWG7zzinGeoT4V7nLRtUVdh_Z4pxP3-pCVbJz-UgA0KRPOd1H16MRUL8sPB0BwjuerLrt9_RsW6QO8GuZql6d2hVBFu24DTx5rDxJLOi-d5cKxQ/s2225/Logo%20Apache%20Spark.png

Behandling av data



Zeppelin Konfigurasjon

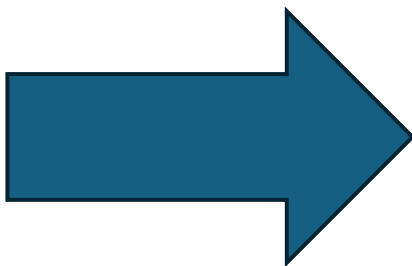


```
spark.conf.set("spark.sql.session.timeZone", "Europe/Oslo")
```

Schema



Kolonnenavn	Datatype	Begrunnelse
fra	TimestampType	ISO-8601[58] format satt til norsk tid. Passer som <i>timestamp</i>
til	TimestampType	ISO-8601[58] format satt til norsk tid. Passer som <i>timestamp</i>
dato	DateType	Datering med format 'dd-MM-yyyy'. Passer som <i>date</i> .
fra tidspunkt	StringType	Klokkeslett med format 'HH:MM'. Defineres som <i>string</i> for å unngå datatap.
til tidspunkt	StringType	Klokkeslett med format 'HH:MM'. Defineres som <i>string</i> for å unngå datatap.
felt	StringType	Varierer mellom numeriske og string verdier i celler. Defineres som <i>string</i> for å unngå datatap.
trafikkmengde	ShortType	Heltallsverdier som ikke overstiger 1000 per celle. Passer som <i>short</i> (range: -32768 → 32767)
dekningsgrad (%)	StringType	Tallverdi med komma som desimal. Defineres som <i>string</i> for å unngå datatap.
antall timer total	ByteType	Heltallsverdier med små verdier. Passer som <i>byte</i> (range: -128 → 127)
7,6m - 12,5m	ShortType	Heltallsverdier som ikke overstiger 1000 per celle. Passer som <i>short</i> (range: -32768 → 32767)



```
%spark
val trafikk_Schema = StructType(Array(
  StructField("trafikkregistreringspunkt", StringType, true),
  StructField("navn", StringType, true),
  StructField("vegreferanse", StringType, true),
  StructField("fra", TimestampType, true),
  StructField("til", TimestampType, true),
  StructField("dato", DateType, true),
  StructField("fra tidspunkt", StringType, true),
  StructField("til tidspunkt", StringType, true),
  StructField("felt", StringType, true),
  StructField("trafikkmengde", ShortType, true),
  StructField("dekningsgrad (%)", StringType, true),
  StructField("antall timer total", ByteType, true),
  StructField("antall timer inkludert", ByteType, true),
  StructField("antall timer ugyldig", ByteType, true),
  StructField("ikke gyldig lengde", ByteType, true),
  StructField("lengdekvalitetsgrad (%)", StringType, true),
  StructField("< 5,6m", ShortType, true),
  StructField(">= 5,6m", ShortType, true),
  StructField("5,6m - 7,6m", ShortType, true),
  StructField("7,6m - 12,5m", ShortType, true),
  StructField("12,5m - 16,0m", ShortType, true),
  StructField(">= 16,0m", ShortType, true),
  StructField("16,0m - 24,0m", ShortType, true),
  StructField(">= 24,0m", ShortType, true)
))
```

Import av Data



- Inkluderer header
- Rekursiv fil import
- Predefinert Schema
- Definerer skille tegn
- Håndterer NULL verdier
- Definerer format for Timestamp data
- Definerer datakilde
- Skiller mellom bil/sykkel med 'klasse'

```
%spark
val bil_raw_df = spark.read
    .option("header", true)
    .option("recursiveFileLookup", true)
    .schema(trafikk_Schema)
    .option("delimiter", ";")
    .option("nullValue", "-")
    .option("timestampFormat", "yyyy-MM-dd'T'HH:mmXXX")
    .csv(bil_base)

val sykkel_raw_df = spark.read
    .option("header", true)
    .option("recursiveFileLookup", true)
    .schema(trafikk_Schema)
    .option("delimiter", ";")
    .option("nullValue", "-")
    .option("timestampFormat", "yyyy-MM-dd'T'HH:mmXXX")
    .csv(sykkel_base)

val weather_raw_df = spark.read
    .option("header", true)
    .option("recursiveFileLookup", true)
    .schema(weather_Schema)
    .option("delimiter", ";")
    .option("nullValue", "-")
    .option("timestampFormat", "dd.MM.yyyy HH:mm")
    .csv(weather_base)

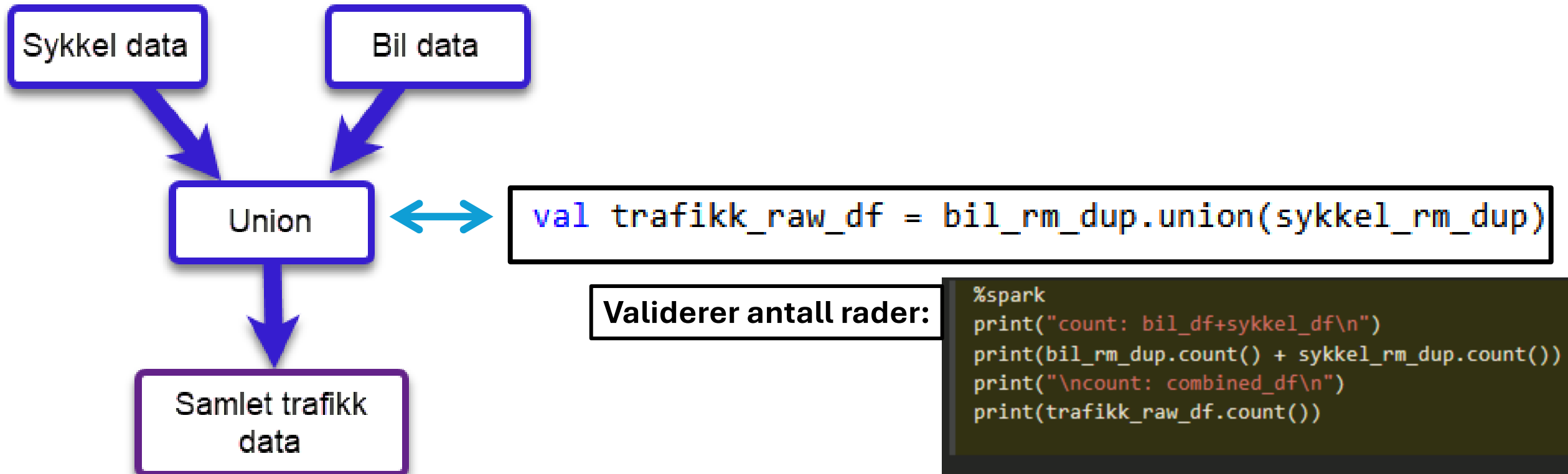
val bil_raw = bil_raw_df.withColumn("klasse", lit(0).cast(ByteType))
val sykkel_raw = sykkel_raw_df.withColumn("klasse", lit(1).cast(ByteType))
```

Fjerne Duplikater



```
val bil_rm_dup = bil_raw.dropDuplicates()  
val sykkel_rm_dup = sykkel_raw.dropDuplicates()  
val weather_rm_dup = weather_raw_df.dropDuplicates()
```

Sammensetting Trafikkdata



Behandling av Data



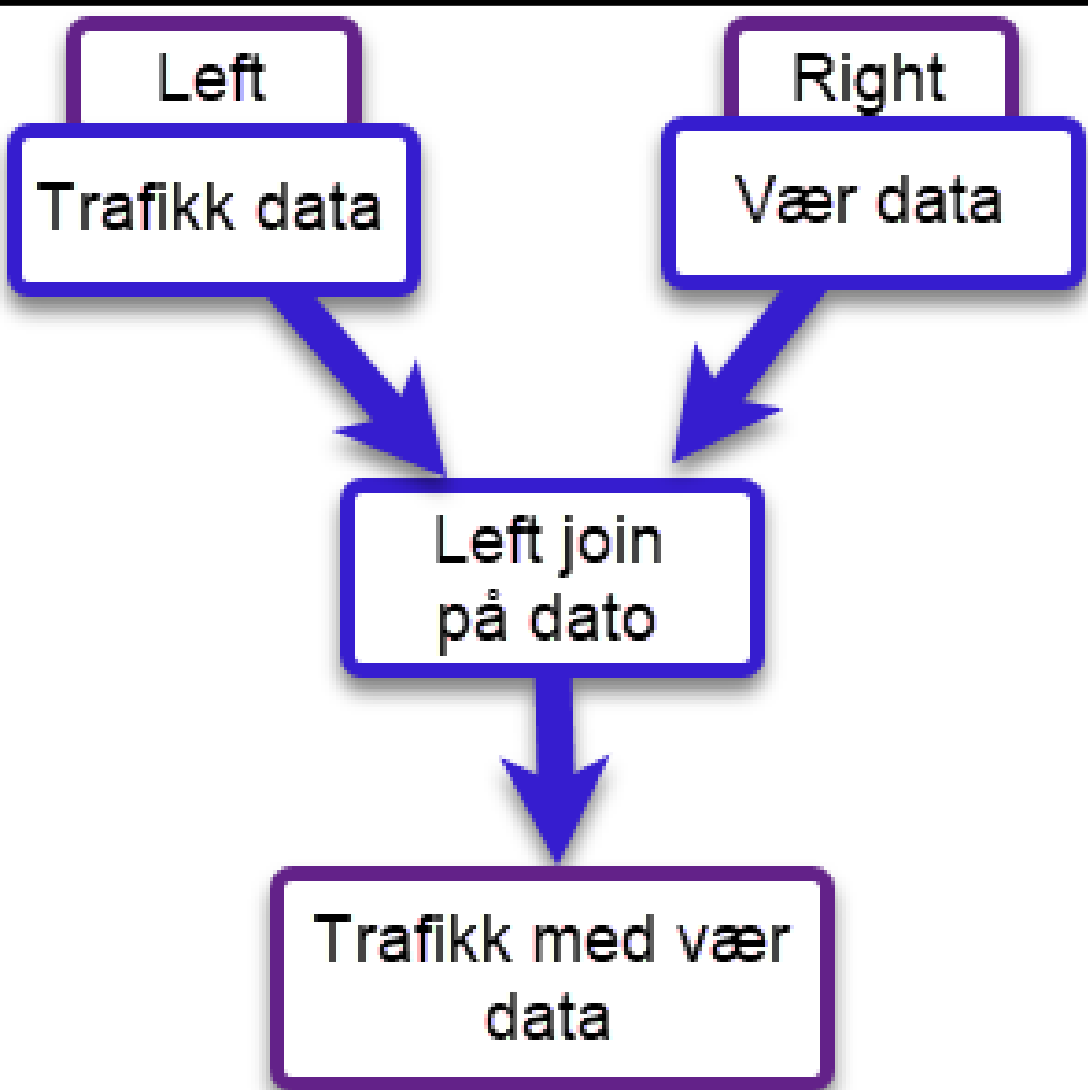
- Desimaler: (, → .) (regexp_replace)
- Fjerne “ sykkel” fra navn (trim(regexp_replace))
- Endre klokkeslett til numerisk verdi
(00:00 → 24) (date_format)
- Cast til korrekt DataType



```
val trafikk_df_formatfix = trafikk_raw_df
  .withColumn(
    "dekningsgrad (%)",
    regexp_replace(col("dekningsgrad (%)"), ",", ".").cast(FloatType)
  )
  .withColumn(
    "lengdekvalitetsgrad (%)",
    regexp_replace(col("lengdekvalitetsgrad (%)"), ",", ".").cast(FloatType)
  )
  .withColumn(
    "navn",
    trim(regexp_replace(col("navn"), "\\b sykkel\\b", ""))
  )
  .withColumn(
    "fra tidspunkt",
    date_format(col("fra"), "k").cast("byte")
  )
  .withColumn(
    "til tidspunkt",
    date_format(col("til"), "k").cast("byte")
  )

val weather_df_formatfix = weather_rm_dup
  .withColumn(
    "nedbør (1 t)",
    regexp_replace(col("nedbør (1 t)"), ",", ".").cast(FloatType)
  )
  .withColumn(
    "lufttemperatur",
    regexp_replace(col("lufttemperatur"), ",", ".").cast(FloatType)
  )
  .withColumn(
    "høyeste middelvind (1 t)",
    regexp_replace(col("høyeste middelvind (1 t)"), ",", ".").cast(FloatType)
  )
```


Sammensetning Trafikk/Vær



```
val complete_df_raw = trafikk_df_formatFix.alias("c")
    .join(weather_df_formatfix.alias("w"),
        col("c.fra") === col("w.tid(norsk normaltid)"), "left")
    .select(
        col("c.*"),
        col("w.lufttemperatur"),
        col("w.høyeste middelvind (1 t)").as("høyeste middelvind (m/s)")
    )
```


Fjerne Upassende/Feil Data



- **Værdata** allerede ferdig behandlet.

- **Trafikkdata:**

- Dekningsgrad (%)

Fjerner der;

dekningsgrad == 0

&&

trafikkmengde > 0

```
val validation = complete_df_raw
    .filter( ! (col("dekningsgrad (%)") === 0
    &&
    col("trafikkmengde").isNotNull))
```

- Lengdekvalitetsgrad (%)

Fjerner der;

lengdekvalitetsgrad == 0

&&

trafikkmengde > 0

&&

Klasse == 0 (biler)

```
val complete_df = validation
    .filter( ! (col("Lengdekvalitetsgrad (%)") === 0
    &&
    col("trafikkmengde").isNotNull
    &&
    col("klasse") === 0))
```

Cassandra



- Hva ønsker vi å hente ut av data for å svare på de individuelle hypotesene?
- Formet tabeller i spark for å vise informasjonen vi ønsket.
- Satte opp tenkte CQL-spørringer for å hente ut ønsket informasjon.
- Designet Cassandra tabeller basert på disse spørringene.
- Testet at spørringene fungerte som tenkte.
- Hvis ikke? Gjennomfør ny iterasjon av de siste 2 stegene.

Cassandra



Hypotese 1:

Den totale trafikkmengden inn og ut av Fredrikstad gjennom én time er høyest i rushtiden hverdager mellom 15 : 00 – 16 : 00.

```
%cassandra
USE capstone_ks;

SELECT * FROM rushtid_1
  WHERE navn = 'Glemmen'
  AND dato >= '2024-03-04'
  AND dato <= '2024-03-10';
```

```
%cassandra
USE capstone_ks;

CREATE TABLE IF NOT EXISTS rushtid_1(
  navn TEXT,
  dato DATE,
  ukedag TEXT,
  fra TINYINT,
  til TINYINT,
  trafikkmengde SMALLINT,
  PRIMARY KEY (navn, dato, fra, trafikkmengde)
) WITH CLUSTERING ORDER BY (dato ASC, fra ASC);
```

```
%spark
val df_rushtid_1 = complete_df
  .select(
    col("navn"),
    col("dato"),
    col("ukedag"),
    col("fra"),
    col("til"),
    col("trafikkmengde")
  )
  .filter(!(col("trafikkmengde").isNull))
  .filter(col("felt") === "Totalt")
  .filter(!col("ukedag").contains("Saturday"))
  .filter(!col("ukedag").contains("Sunday"))
  .withColumn(
    "fra",
    date_format(col("fra"), "k").cast("byte")
  )
  .withColumn(
    "til",
    date_format(col("til"), "k").cast("byte")
  )

df_rushtid_1.write
  .format("org.apache.spark.sql.cassandra")
  .options(Map(
    "table" -> "rushtid_1",
    "keyspace" -> "capstone_ks"
  ))
  .mode("append")
  .save()
```

Cassandra



Hypotese 2:

Den totale bil trafikkmengden inn og ut av Fredrikstad er lavere i juli måned enn for resten av året.

```
%spark
val df_tot_traffic_month = complete_df
  .select(
    col("navn"),
    col("dato"),
    col("ukedag"),
    col("trafikkmengde")
  )
  .filter(!(col("trafikkmengde").isNull))
  .filter(col("felt") === "Totalt")
  .withColumn("mnd", month(col("dato")).cast("byte"))
  .withColumn("yr", date_format(col("dato"), "yyyy").cast("int"))
  .groupBy("yr", "mnd", "navn")
  .agg(sum(col("trafikkmengde")).as("trafikkmengde"))
  .orderBy("yr", "mnd")

df_tot_traffic_month.write
  .format("org.apache.spark.sql.cassandra")
  .options(Map(
    "table" -> "tot_traffic_month",
    "keyspace" -> "capstone_ks"
  ))
  .mode("append")
  .save()
```

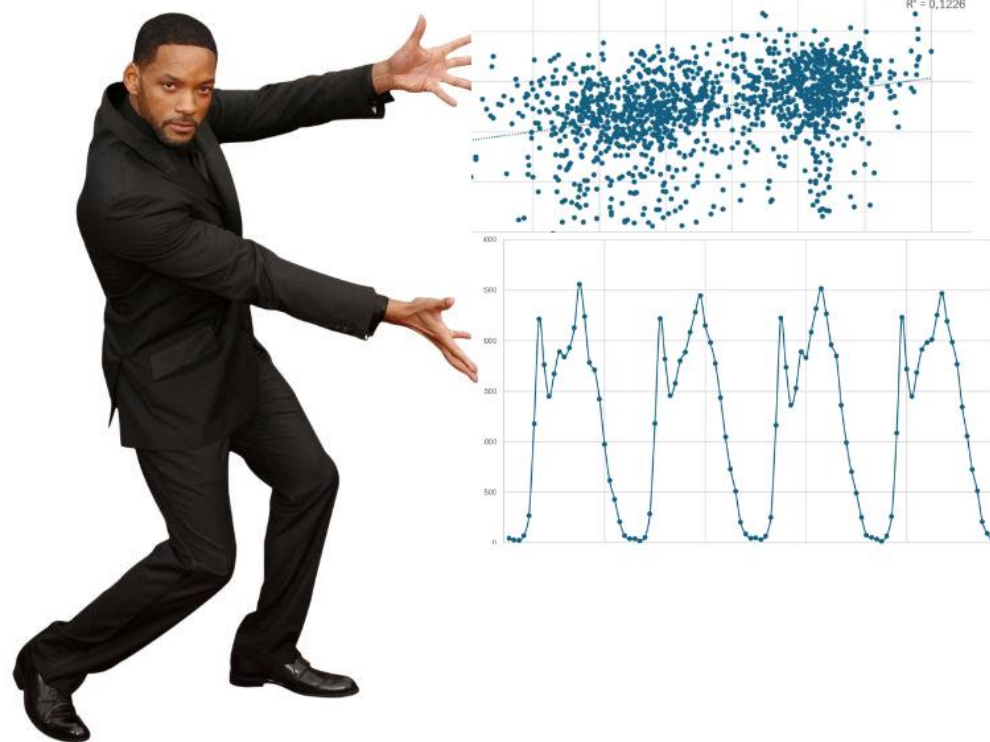
```
%cassandra
USE capstone_ks;

SELECT * FROM tot_traffic_month
WHERE navn = 'Glemmen'
AND yr >= 2020;
```

```
%cassandra
USE capstone_KS;

CREATE TABLE IF NOT EXISTS tot_traffic_month(
  yr INT,
  mnd TINYINT,
  navn TEXT,
  trafikkmengde BIGINT,
  PRIMARY KEY (navn, yr, mnd, trafikkmengde)
) WITH CLUSTERING ORDER BY (yr ASC, mnd ASC);
```

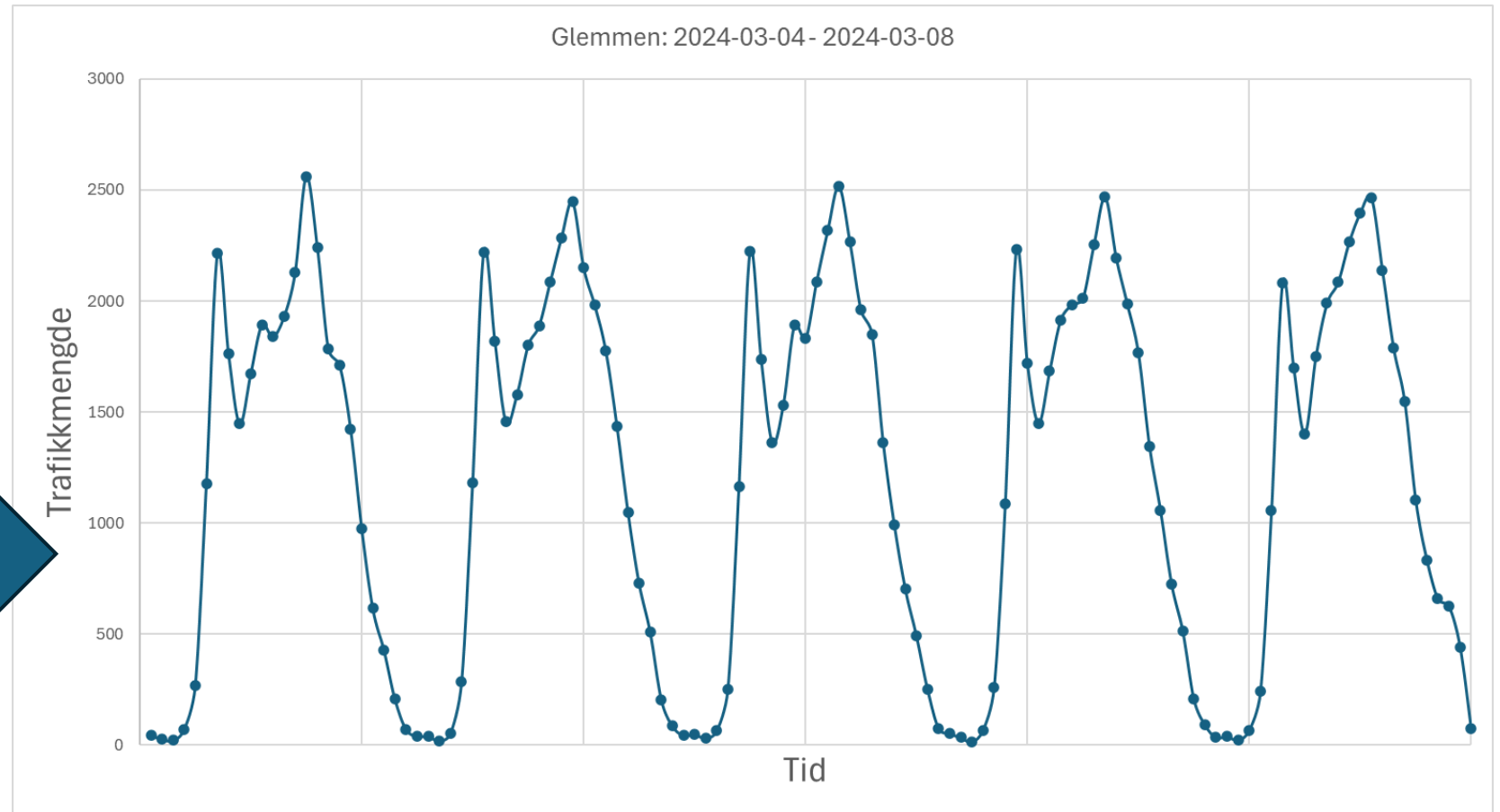
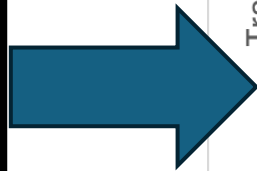
Resultater



Resultater: Hypotese 1

Den totale trafikkmengden inn og ut av Fredrikstad gjennom én time er høyest i rushtiden hverdager mellom 15 : 00 – 16 : 00.

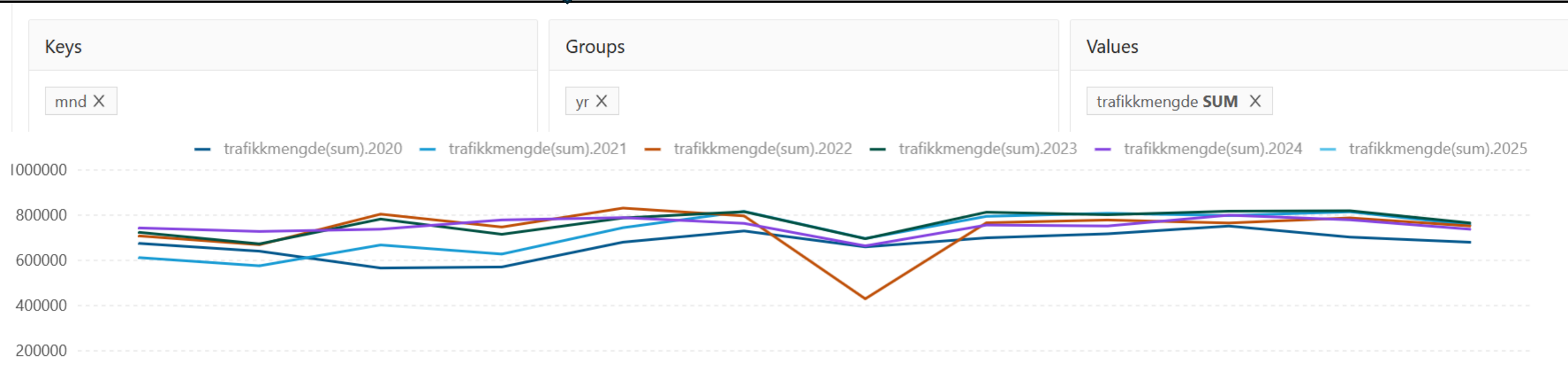
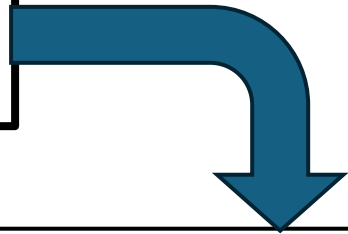
```
SELECT * FROM rushtid_1  
WHERE navn = 'Glemmen'  
AND dato > '2024-03-04'  
AND dato < '2024-03-08';
```



Resultater: Hypotese 2

```
%cassandra  
USE capstone_ks;  
  
SELECT * FROM tot_traffic_month  
WHERE navn = 'Glemmen'  
AND yr >= 2020;
```

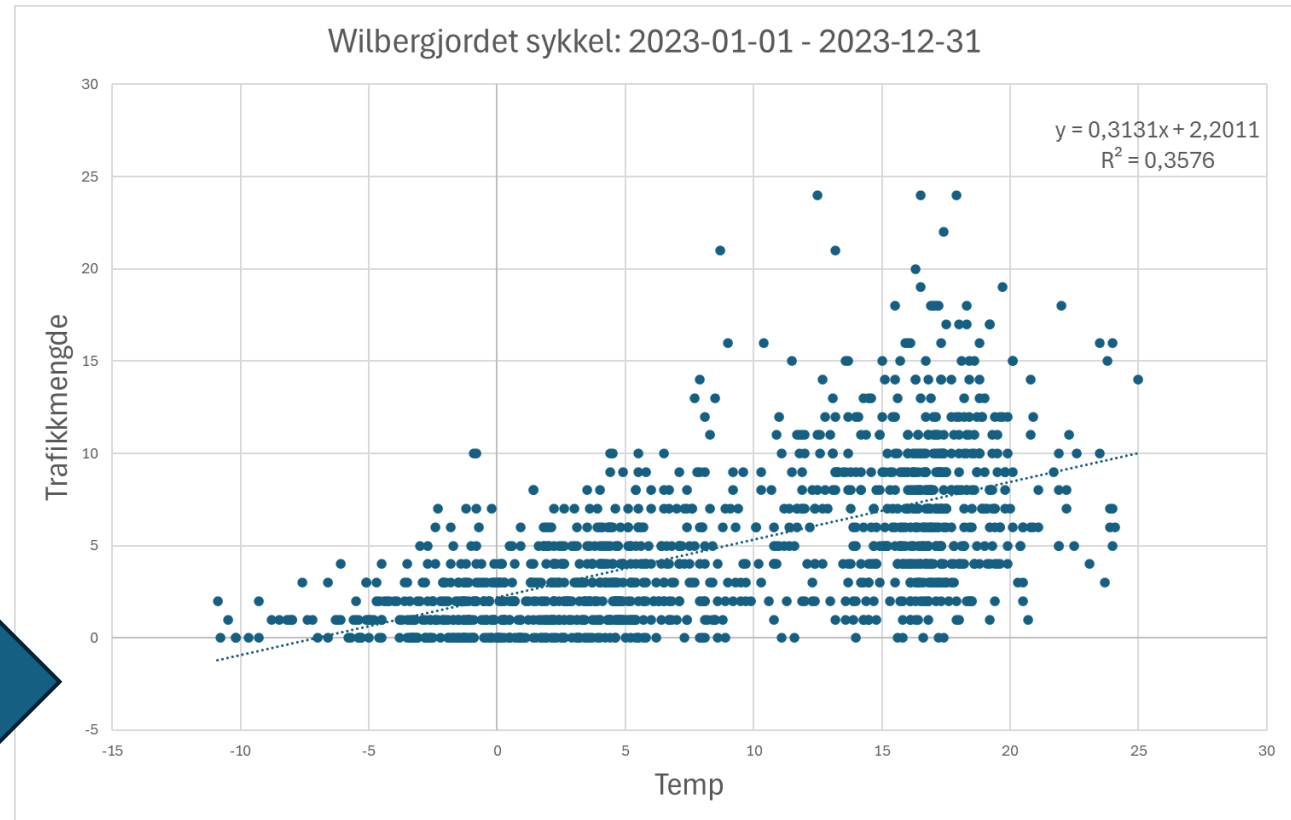
Den totale bil trafikkmengden inn og ut av Fredrikstad er lavere i juli måned enn for resten av året.



Resultater: Hypotese 3

Det finnes en positiv korrelasjon for temperatur og total sykkeltrafikk i Fredrikstad målt over en referanseperiode mellom 10:00 – 14:00.

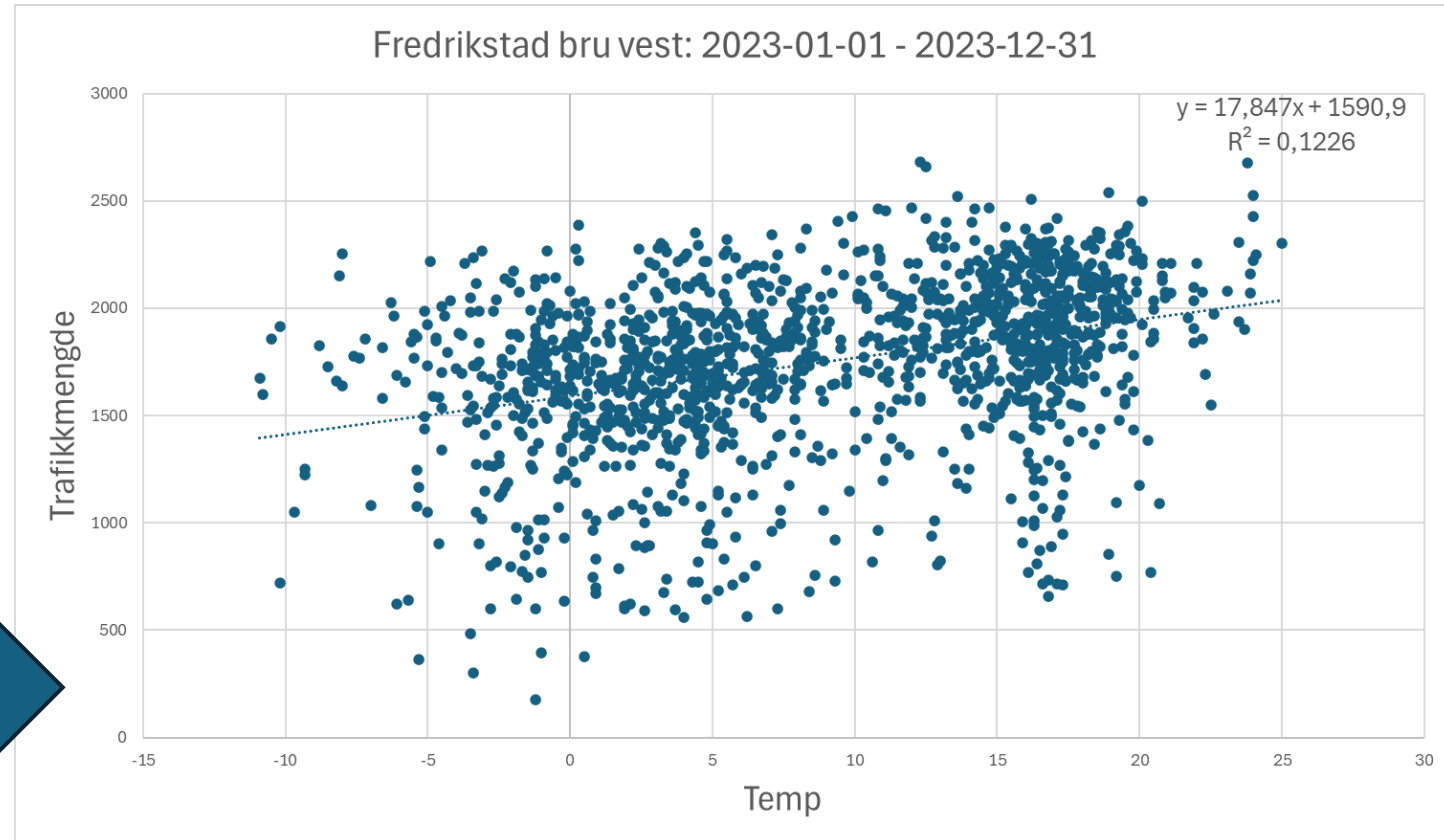
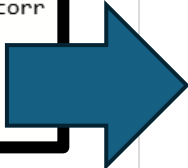
```
SELECT navn, dato, fra, trafikkmengde, lufttemperatur FROM sykkel_temp_corr  
WHERE navn = 'Wilbergjordet sykkelstavle mot metring'  
AND dato >= '2023-01-01'  
AND dato < '2024-01-01';
```



Resultater: Hypotese 4

Det finnes en negativ korrelasjon for temperatur og total biltrafikk i Fredrikstad målt over en referanseperiode mellom 10:00 – 14:00.

```
SELECT navn, dato, fra, trafikkmenge, lufttemperatur FROM bil_temp_corr  
WHERE navn = 'Fredrikstad bru vest'  
AND dato >= '2023-01-01'  
AND dato < '2024-01-01';
```



Takk for oss!